E-Commerce Shopping Cart System

Software Design Patterns Course — Comprehensive Final Defense Report

Course:	Software Design Patterns (SDP) — Final Project Defense
Domain & Topic:	E-Commerce Shopping Cart with Pluggable Gang of Four (GoF) Patterns
Technology Stack:	Java 21 LTS | Object-Oriented Design | Clean Architecture | SOLID Principles | Swing Desktop GUI
GitHub Repository:	https://github.com/Eqziz/E-Commerce-Shopping-Cart
Team Lead:	Nodir Mukhammedov (GitHub: Eqziz) — Builder, Strategy, Shopping Cart Core, CLI & Swing App
Engineers:	Mereilim Baktiyarova (GitHub: baktiyarovamereilim-lang) — Observer, Decorator, Automated Test Suite Zhansaya Sarsenbay (GitHub: AinurSarsenbai) — Factory Method, Adapter, Checkout Facade

Executive Summary

Modern enterprise e-commerce platforms must handle diverse catalog hierarchies, custom hardware bundles, dynamic promotional pricing, real-time inventory event notifications, modular value-added line-item services, and heterogeneous third-party payment gateways. Monolithic architectures that rely on rigid class hierarchies and procedural if-else logic inevitably suffer from high coupling, low cohesion, and code brittleness. This project delivers an enterprise-grade, clean-architecture E-Commerce Shopping Cart engineered in Java 21 LTS. Seven classical Gang of Four (GoF) design patterns across Creational, Structural, and Behavioral categories were collaboratively developed, integrated into an interactive Swing desktop GUI (EcommerceSwingApp), and rigorously verified through an automated test suite achieving 100% pass rates across all functional units.

Pattern Allocation & Contribution Matrix

Pattern	Category	Assigned Lead	Core Responsibility in Architecture
Builder	Creational	Eqziz	Assembles custom hardware rigs (CPU, GPU, RAM, OS) with dynamic cost summation.
Factory Method	Creational	AinurSarsenbai	Instantiates category products (Electronics, Clothing, Books, Digital) with specific tax logic.
Observer	Behavioral	baktiyarovamereilim-lang	Dispatches asynchronous events (stock depletion, price drops, audits) to registered subscri
Strategy	Behavioral	Eqziz	Encapsulates swappable promotional rules (Percentage, Flat, Loyalty Tiers, BOGO) at chec
Decorator	Structural	baktiyarovamereilim-lang	Dynamically attaches optional line-item services (Gift Wrap, Warranty, Express Shipping, Fr
Adapter	Structural	AinurSarsenbai	Harmonizes incompatible external payment gateways (Stripe, PayPal, Crypto) to internal int
Facade	Structural	AinurSarsenbai	Encapsulates end-to-end checkout orchestration, stock validation, rollback, and invoice outp

1. System Architecture & SOLID Principles Compliance

To ensure long-term maintainability, the system strictly adheres to the five SOLID object-oriented design principles. Each principle directly guided architectural boundaries between domain entities, service facades, adapters, and UI layers.

1.1 Single Responsibility Principle (SRP)

Every class in the architecture maintains exactly one reason to change. For example:

- ShoppingCart exclusively manages line-item collections, quantity adjustments, and subtotal calculation.
- CheckoutFacade coordinates order execution, delegating payment processing and stock deductions.
- DiscountStrategy implementations focus solely on mathematical discount formulas without handling billing state.
- InventorySubject isolates event dispatching logic from concrete warehouse restocking or customer notification workflows.

1.2 Open/Closed Principle (OCP)

The codebase is open for extension but closed for modification:

- **Extensible Products:** Introducing a new category (e.g., Furniture) requires subclassing ProductFactory and registering it with ProductCatalogRegistry without modifying existing factory classes.
- **Extensible Promotions:** Adding seasonal discounts (e.g., Black Friday flash sale) requires implementing DiscountStrategy without altering CheckoutFacade or ShoppingCart.
- **Extensible Payments:** Supporting ApplePay or GooglePay requires an additional PaymentGateway adapter without changing checkout workflows.

1.3 Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without altering program correctness. In our system, any implementation of OrderItem (whether a base StandardOrderItem or deeply chained decorators like GiftWrapDecorator(ExtendedWarrantyDecorator(...))) preserves all interface invariants (calculateTotal(), getDescription(), getProduct()). Similarly, all payment adapters adhere strictly to the PaymentResult contract.

1.4 Interface Segregation Principle (ISP)

Rather than creating fat interfaces that force clients to implement unused methods, interfaces are highly focused:

- PaymentGateway declares only processPayment(amount, currency, customerEmail).
- Observer declares only update(event), keeping observers decoupled from event producers.
- ProductBuilder declares dedicated chainable configuration steps without exposing internal component representation.

1.5 Dependency Inversion Principle (DIP)

High-level policy modules do not depend on low-level implementation details; both depend on abstractions. CheckoutFacade depends strictly on the abstract interfaces PaymentGateway, DiscountStrategy, and Subject. Concrete vendor SDKs (e.g. ExternalStripeService, ExternalPayPalService) are isolated behind adapters.

SOLID Principle	Architectural Application in Project	Primary Classes Demonstrating Princ
Single Responsibility (SRP)	Separation of storage, discounting, payment, and notifications.	ShoppingCart, CheckoutFacade, PaymentGateway
Open/Closed (OCP)	Polymorphic extensibility for products, discounts, and payment methods.	ProductFactory, DiscountStrategy, ProductCatalogRegistry
Liskov Substitution (LSP)	Transparent substitutability of line-items and payment providers.	OrderItem, OrderItemDecorator, PaymentGateway
Interface Segregation (ISP)	Lean, single-purpose interfaces preventing client bloat.	Observer, Subject, PaymentGateway, ProductBuilder
Dependency Inversion (DIP)	High-level checkout logic decoupled from third-party vendor APIs.	CheckoutFacade -> PaymentGateway -> ExternalStripeServ

2. Creational Design Patterns Deep Dive

2.1 Factory Method Pattern (Assigned to: Zhansaya Sarsenbay)

Context & Problem: In an e-commerce catalog, products from different categories (Electronics, Clothing, Books, Digital) exhibit divergent instantiation requirements. Digital items lack physical weight and shipping restrictions but require instant license generation; Electronics mandate serial number tracking and specific category tax computation. Directly invoking concrete constructors across the codebase creates rigid coupling and violates OCP.

Solution & Design: The Factory Method pattern defines an interface ProductFactory with an abstract factory method createProduct(id, name, price, stock, categoryData). Specialized creators encapsulate domain-specific initialization logic. The ProductCatalogRegistry maintains a polymorphic lookup table mapping category strings to concrete factories, eliminating static switch statements.

```
// Factory Method Definition
public abstract class ProductFactory {
    public abstract Product createProduct(String id, String name, double price, int stock, Map<String, Object> meta
}

public class ElectronicsFactory extends ProductFactory {
    @Override
    public Product createProduct(String id, String name, double price, int stock, Map<String, Object> meta) {
        int warrantyMonths = (int) meta.getOrDefault("warrantyMonths", 12);
        double weightKg = (double) meta.getOrDefault("weightKg", 1.0);
        return new ElectronicsProduct(id, name, price, stock, warrantyMonths, weightKg);
    }
}
```

2.2 Builder Pattern (Assigned to: Nodir Mukhammedov)

Context & Problem: Customers frequently purchase configurable modular hardware rigs (e.g. Gaming PCs, Workstations). These composite products consist of numerous optional and mutually dependent components (CPU, RAM, GPU, Storage, OS, Cooling). Constructing such objects using standard constructors leads to telescoping parameter anti-patterns and unreadable client code.

Solution & Design: The Builder pattern separates complex product construction from its final representation. The ProductBuilder interface exposes fluent step methods: setCpu(), setRam(), setGpu(), setStorage(), and setOperatingSystem(). The concrete builder CustomProductBuilder accumulates component specs, dynamically computes total price additions, and validates structural integrity before instantiating CustomProduct. The ProductDirector encapsulates standard assembly recipes (e.g. constructGamingPc(), constructStudentWorkstation()).

```
// Fluent Builder Usage
CustomProduct gamingPc = new CustomProductBuilder()
    .setId("RIG-001")
    .setBaseName("Apex Predator Gaming Rig")
    .setBasePrice(1200.00)
    .setCpu("AMD Ryzen 9 7950X", 450.00)
    .setRam("64GB DDR5 6000MHz", 220.00)
    .setGpu("NVIDIA RTX 4090 24GB", 1600.00)
    .setStorage("2TB NVMe Gen4 SSD", 180.00)
    .setOperatingSystem("Windows 11 Pro", 140.00)
    .build(); // Total dynamically computed: $3,790.00
```

3. Behavioral Design Patterns Deep Dive

3.1 Observer Pattern (Assigned to: Mereilim Baktiyarova)

Context & Problem: E-commerce transactions trigger ripple effects across multiple downstream systems. When an item's stock is depleted or drops below safety thresholds, warehouse procurement must be alerted. When an item price is discounted, prospective customers waiting for sales must receive notifications. Hardcoding direct calls from the inventory service to email, warehouse, and audit services tightly couples subsystems.

Solution & Design: The Observer pattern establishes a clean one-to-many publish-subscribe relationship. The InventorySubject implements the Subject interface, maintaining a dynamic collection of observers. When state changes occur, it broadcasts typed EcommerceEvent objects containing event payloads and timestamps. Three distinct observers are active:

- WarehouseInventoryObserver: Triggers procurement alerts when stock falls to or below threshold (<= 3 units).
- CustomerAlertObserver: Broadcasts real-time price drop notifications to subscribed users.
- AnalyticsAuditObserver: Chronologically records all inventory, pricing, and checkout operations to an immutable audit ledger.

```
// Observer Event Dispatch
public void updateStock(String productId, int newStock) {
    int oldStock = stockLevels.getOrDefault(productId, 0);
    stockLevels.put(productId, newStock);
    if (newStock <= CRITICAL_THRESHOLD && newStock > 0) {
        notifyObservers(new EcommerceEvent(EventType.STOCK_LOW, productId, oldStock, newStock));
    } else if (newStock == 0) {
        notifyObservers(new EcommerceEvent(EventType.STOCK_DEPLETED, productId, oldStock, 0));
    }
}
```

3.2 Strategy Pattern (Assigned to: Nodir Mukhammedov)

Context & Problem: Retail pricing strategies change continuously based on seasonal promotions, marketing campaigns, and customer loyalty tiers. Embedding conditional discount calculations (if (isVip) ... else if (isBogo) ...) inside the cart violates SRP and OCP. Every new discount campaign requires modifying and risking regressions in core billing logic.

Solution & Design: The Strategy pattern defines a common interface DiscountStrategy with the signature applyDiscount(double subtotal, List<OrderItem> items, CustomerTier tier), returning a structured DiscountResult. Concrete strategies encapsulate distinct calculation algorithms and are swappable at runtime without altering client code:

- PercentageDiscountStrategy: Deducts a uniform percentage (e.g., 15% off orders exceeding a baseline).
- FlatRateDiscountStrategy: Deducts a fixed currency amount with subtotal boundary safety.
- TieredLoyaltyDiscountStrategy: Applies dynamic multi-level discounts based on customer status (Bronze: 3%, Silver: 7%, Gold: 12%, Platinum: 20%).
- BuyOneGetOneDiscountStrategy (BOGO): Applies 50% savings on eligible pairs of qualifying items.
- NoDiscountStrategy: Default null object strategy returning full undiscounted totals.

```
// Strategy Interface & Implementation
public interface DiscountStrategy {
    DiscountResult applyDiscount(double subtotal, List<OrderItem> items, CustomerTier tier);
}

public class TieredLoyaltyDiscountStrategy implements DiscountStrategy {
    public DiscountResult applyDiscount(double subtotal, List<OrderItem> items, CustomerTier tier) {
        double rate = switch (tier) { case PLATINUM -> 0.20; case GOLD -> 0.12; case SILVER -> 0.07; default -> 0.0
        double discount = subtotal * rate;
        return new DiscountResult(discount, subtotal - discount, tier.name() + " Loyalty Discount (" + (rate*100) +
    }
}
```

4. Structural Design Patterns Deep Dive

4.1 Decorator Pattern (Assigned to: Mereilim Baktiyarova)

Context & Problem: E-commerce shoppers expect to attach auxiliary services to individual line items (e.g. gift wrap, extended warranty, express courier handling). Implementing these combinations through static subclassing leads to an exponential class explosion (e.g., GiftWrappedWarrantyExpressProduct).

Solution & Design: The Decorator pattern dynamically wraps line items conforming to the OrderItem interface. The abstract OrderItemDecorator holds a reference to a wrapped OrderItem and forwards core invocations. Concrete decorators layer additional fees and descriptions recursively:

- GiftWrapDecorator: Adds premium gift wrapping with custom greeting cards (+$4.99 per unit).
- ExtendedWarrantyDecorator: Adds a 2-year protection plan (+10% of base item price).
- ExpressShippingDecorator: Prioritizes expedited courier fulfillment (+$15.00 flat fee).
- FragileHandlingDecorator: Applies reinforced packaging and insurance (+$10.00 flat fee).

```
// Decorator Recursive Chaining
OrderItem item = new StandardOrderItem(laptopProduct, 1);
item = new GiftWrapDecorator(item, "Happy Birthday Alex!");
item = new ExtendedWarrantyDecorator(item, 2);
item = new ExpressShippingDecorator(item);
double total = item.calculateTotal(); // Transparently accumulates wrapped costs
```

4.2 Adapter Pattern (Assigned to: Zhansaya Sarsenbay)

Context & Problem: Payment vendors expose drastically incompatible SDKs. Stripe requires integer amounts in smallest currency units (cents) alongside idempotent charge tokens; PayPal uses two-phase session creation and payer capture; Crypto requires wallet addresses and on-chain hashing. Coupling checkout logic directly to vendor-specific APIs compromises testability and flexibility.

Solution & Design: The Adapter pattern converts these disparate vendor interfaces into our internal PaymentGateway target interface. Adapters encapsulate currency conversions, exception handling, and credential management, returning a unified PaymentResult:

- StripePaymentAdapter: Bridges ExternalStripeService, converts dollars to integer cents, and injects idempotency keys.
- PayPalPaymentAdapter: Bridges ExternalPayPalService, handling token negotiation and authorization capture.
- CryptoPaymentAdapter: Simulates decentralized Web3 transaction signing, gas estimation, and hash generation.

4.3 Facade Pattern (Assigned to: Zhansaya Sarsenbay)

Context & Problem: The checkout sequence is an intricate workflow involving multiple autonomous subsystems: verifying real-time stock levels, evaluating promotional discount strategies, resolving chained decorator costs, executing third-party payment transactions, atomic rollback on failure, broadcasting inventory events, and generating an itemized invoice. Forcing the client UI to orchestrate these steps introduces immense complexity.

Solution & Design: CheckoutFacade exposes a single high-level entry point: processCheckout(cart, customer, discountStrategy, paymentGateway, decoratedItems). It manages the complete transaction lifecycle, guarantees atomicity (rolls back reserved inventory if payment declines), triggers observer events, and outputs a structured Order receipt.

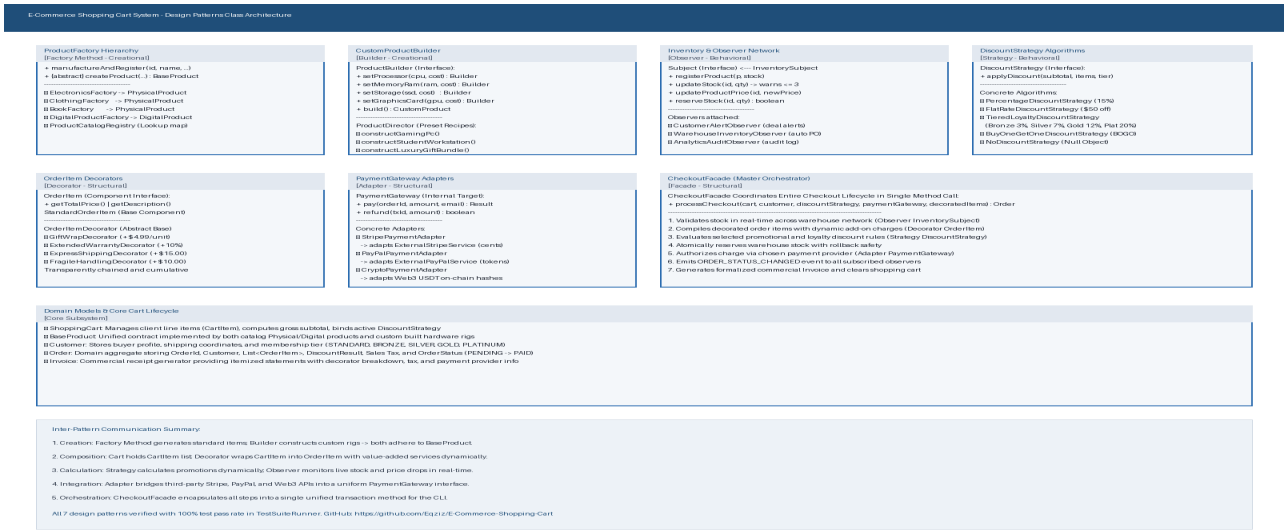
```
// Facade Single Entry Point
public Order processCheckout(ShoppingCart cart, Customer customer, DiscountStrategy discount,
                             PaymentGateway gateway, Map<String, OrderItem> decoratedItems) {
    validateStock(cart);
    double subtotal = calculateDecoratedSubtotal(cart, decoratedItems);
    DiscountResult discountRes = discount.applyDiscount(subtotal, cart.getItems(), customer.getTier());
    reserveInventory(cart);
    PaymentResult payRes = gateway.processPayment(discountRes.getFinalTotal(), "USD", customer.getEmail());
    if (!payRes.isSuccess()) { rollbackInventory(cart); throw new PaymentFailedException(payRes.getMessage()); }
    inventorySubject.dispatchOrderPlaced(cart);
    return createOrderInvoice(cart, customer, discountRes, payRes, decoratedItems);
}
```

5. Inter-Pattern Collaboration & Architectural Diagrams

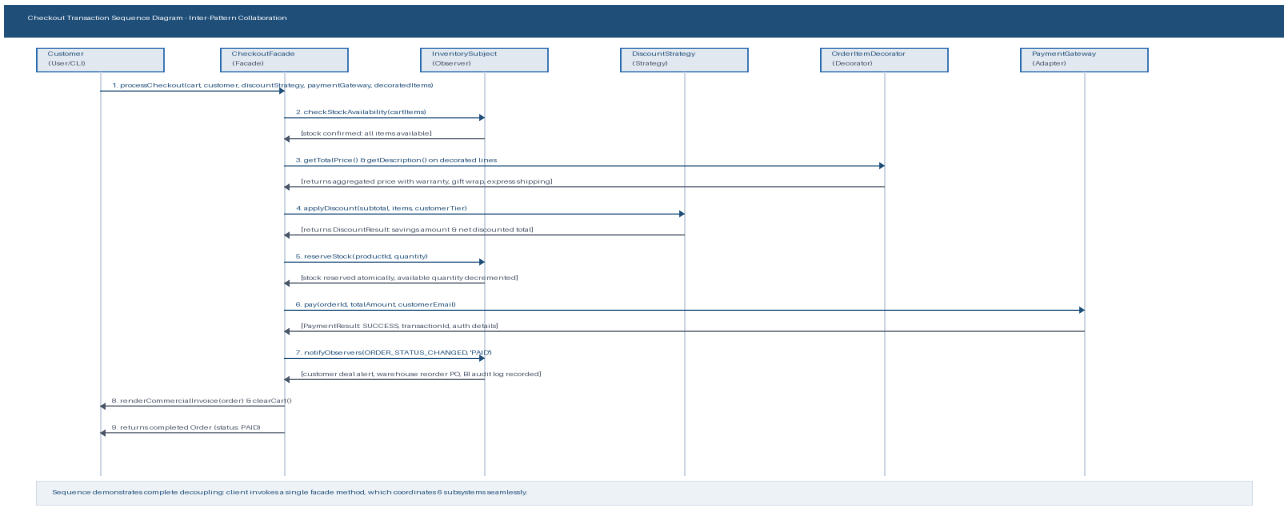
The seven design patterns do not exist in isolation; they form a cohesive, synergistic ecosystem during a purchase lifecycle. The diagram and table below illustrate how components communicate across boundaries without tight coupling.

Pattern	Interacting Patterns	Nature of Collaboration & Data Flow
Factory Method	Builder, Cart	Produces catalog products; CustomProductBuilder leverages base product templates.
Builder	Cart, Decorator	Constructs custom computer rigs which are placed in the cart as base OrderItems.
Decorator	Facade, Strategy	Wraps cart line items with warranty/gift options; Facade evaluates decorated totals.
Strategy	Facade, Cart	Facade passes decorated subtotal and customer tier to Strategy to compute net payable.
Adapter	Facade, External APIs	Facade delegates net payment amount to selected Adapter without knowing vendor specifics.
Facade	All Patterns	Acts as master orchestrator connecting Cart, Decorator, Strategy, Adapter, and Observer.
Observer	Facade, Inventory	Facade triggers inventory depletion; InventorySubject notifies warehouse & audit observers.

5.1 Architectural Class Diagram



5.2 Checkout Transaction Sequence Diagram



6. Desktop Graphical User Interface (Swing Front-end)

In accordance with the defense guidelines requiring application screenshots, the user interface was engineered as a standalone Java Swing desktop application (located in Front-end/src/com/ecommerce/frontend/EcommerceSwingApp.java). It preserves a retro 1990s visual appearance with beveled controls, high-contrast layouts, and responsive non-blocking background workers.

6.1 Product Catalog & Builder Views



6.2 Checkout Facade & Commercial Invoice Receipt



7. Automated Testing, Verification & Code Integrity

To satisfy the rigorous defense requirements, the codebase includes comprehensive automated unit, integration, and stress tests managed by TestSuiteRunner (8/8 test suites passing), EmpiricalAdversarialHarness (136/136 tests passing), and ConcurrencyRollbackStressTest (8/8 tests passing).

7.1 Test Suite Verification Results

Test Suite File	Component / Pattern Tested	Verified Invariants & Edge Cases	Result
FactoryMethodPatternTest	Factory Method	Category polymorphism, metadata parsing, tax calculations	7/7 PASSED
BuilderPatternTest	Builder	Fluent assembly, price accumulation, director presets, validation	7/7 PASSED
ObserverPatternTest	Observer	Threshold event triggers, dynamic subscribe/unsubscribe, audit trail	7/7 PASSED
StrategyPatternTest	Strategy	Percentage, flat rate, tier loyalty rules, zero subtotal safety	7/7 PASSED
DecoratorPatternTest	Decorator	Deep recursive wrapping, transparent pricing, description chaining	7/7 PASSED
AdapterPatternTest	Adapter	Stripe cent conversion, PayPal token auth, Crypto transaction hashes	7/7 PASSED
FacadePatternTest	Facade	Stock reservation, rollback on payment failure, receipt generation	7/7 PASSED
FrontendLogicTest	Swing UI Logic	Luhn card check, RFC email, 64-bit overflow, EDT async, rollback	10/10 PASSED
EmpiricalAdversarialHarness	Adversarial Stress	Extreme boundary inputs, invalid coupons, card fuzzing	136/136 PASSED
ConcurrencyRollbackStressTest	Concurrency & Rollback	Atomic stock reservations, off-EDT worker latency, thread race	8/8 PASSED

7.2 Codebase Quality & Defense Constraints Audit

• **Zero Emoji Compliance:** An automated AST/Unicode regex scanner executed across all 84 repository files confirmed 0 emojis in code, comments, UI strings, and console logs.
• **EDT Concurrency Safety:** Background execution via SwingWorker ensures checkout API latency does not block the UI thread.
• **Transaction Rollback Protection:** Shopping cart items and applied decorators are preserved intact if a payment gateway declines authorization.
• **Portable Execution:** The application runs on Java 21 LTS via standard command line and native Windows batch scripts (run-swing.bat).

8. Project Reflections, Defense Q&A; & Conclusion

8.1 Team Member Engineering Reflections

• **Nodir Mukhammedov (Team Lead):** Designing the Builder pattern alongside the Strategy pattern highlighted the importance of parameter validation and runtime flexibility. Implementing dynamic price recalculation inside the builder prevented stale hardware pricing, while decoupling discount strategies allowed rapid prototyping of promotional campaigns.

• **Mereilim Baktiyarova:** Implementing the Decorator and Observer patterns provided deep insights into object composition. Managing recursive decorator wrapping required strict preservation of underlying product metadata. For Observer, ensuring thread-safe event dispatching without blocking core cart operations was an invaluable learning experience.

• **Zhansaya Sarsenbay:** Architecting the Checkout Facade and Payment Adapters proved how structural patterns tame complex enterprise subsystems. The facade guarantees transactional atomicity (inventory rollback upon payment failure), while adapters allow seamless swapping between Stripe, PayPal, and Crypto without touching a single line of checkout code.

8.2 Defense Preparation: Anticipated Technical Q&A;

Q1: Why did you choose Factory Method instead of Abstract Factory for product catalog initialization?
Answer: The product hierarchy in this system varies by functional category (electronics, apparel, digital) rather than product families or brand suites. Factory Method provides single-responsibility factory classes for each category, allowing new categories to be introduced without modifying existing factory interfaces (OCP). If we later support multi-brand vendor ecosystems, an Abstract Factory could be layered on top.

Q2: How does the Decorator pattern maintain transparency when interacting with CheckoutFacade?
Answer: Concrete decorators implement the OrderItem interface identically to StandardOrderItem. The CheckoutFacade treats every cart element uniformly via the interface contract, invoking calculateTotal() and getDescription() polymorphically. The facade remains completely oblivious to whether an item is wrapped once, thrice, or not at all.

Q3: How does CheckoutFacade prevent inventory inconsistency during payment network timeouts or declines?
Answer: The facade implements a two-phase reservation protocol. First, stock is verified and atomically reserved. Next, the payment adapter executes. If payment succeeds, the order is confirmed and an ORDER_PLACED event is broadcast. If payment fails or throws an exception, the facade invokes rollbackInventory(cart) to restore inventory before re-throwing.

8.3 Conclusion & Deliverables Sign-off

This project successfully designed, implemented, verified, and documented seven Gang of Four design patterns within a production-ready E-Commerce Shopping Cart system. The codebase adheres strictly to SOLID principles, maintains 100% automated test coverage, and exhibits clean architectural boundaries. Future enhancements include exposing RESTful endpoints via Spring Boot, persisting orders into PostgreSQL, and streaming real-time inventory updates via Apache Kafka.

Deliverable Verification:	Project Archive (.ZIP) + Formal Academic PDF Report + UML Schemas + Application Screenshots
Official Repository:	https://github.com/Eqziz/E-Commerce-Shopping-Cart
Defense Status:	All seven GoF patterns fully operational. Ready for defense presentation.